

WEB CRAWLER CONCORRENTE EM GO

Wiki Técnica do Projeto

Disciplina:

Paradigmas de Linguagens de Programação

Linguagem:

Go 1.25+

Versão do Documento:

1.0 — Julho 2026

1. Visão Geral

O **Web Crawler Concorrente em Go** é um programa de linha de comando desenvolvido como projeto da disciplina de **Paradigmas de Linguagens de Programação**. A partir de uma URL semente, o crawler navega de forma automática pelas páginas da web, extrai links HTML e continua a exploração até atingir uma profundidade máxima configurável.

O objetivo principal do projeto é demonstrar na prática o modelo de concorrência de Go: **goroutines**, **channels**, **worker pool** e o paradigma CSP (**Communicating Sequential Processes** — 'compartilhe memória comunicando').

2. Funcionalidades

- **Crawl recursivo com profundidade configurável:** navega N níveis a partir da URL inicial.
- **Worker pool concorrente:** múltiplos workers processam URLs em paralelo.
- **Rate limiter global:** controla o número de requisições por segundo para não sobrecarregar o servidor.
- **Deduplicação thread-safe:** cada URL é visitada no máximo uma vez, mesmo com muitos workers simultâneos.
- **Encerramento gracioso:** Ctrl+C cancela a operação de forma ordenada via context.
- **Estatísticas ao final:** exibe URLs descobertas, páginas baixadas, links encontrados e erros.
- **Detector de data races:** compatível com `go run -race` para validação de concorrência.

3. Requisitos

Requisito	Versão / Detalhes
Go	1.25 ou superior
golang.org/x/net/html	Obtida via <code>go mod tidy</code>
Conexão com a internet	Necessária para o crawl
Sistema Operacional	Linux, macOS ou Windows

4. Instalação e Execução

4.1 Clonando e baixando dependências

```
go mod tidy # baixa a dependência golang.org/x/net/html
```

4.2 Executando diretamente

```
go run ./cmd -url=https://www.unb.br/ -depth=2 -workers=5
```

4.3 Compilando o binário

```
go build -o crawler ./cmd  
./crawler -url=https://www.unb.br/ -depth=2 -workers=5
```

4.4 Executando com detector de data races

```
go run -race ./cmd -url=https://www.unb.br/ -depth=1 -workers=50
```

Para encerrar a execução a qualquer momento, pressione **Ctrl+C**.

5. Flags de Configuração

Flag	Padrão	Obrigatório	Descrição
-url	(nenhum)	Sim	URL inicial (semente) do crawl
-depth	2	Não	Profundidade máxima de navegação
-workers	5	Não	Número de workers concorrentes
-rate	10	Não	Limite de requisições por segundo (global)

6. Estrutura do Projeto

Arquivo / Pacote	Responsabilidade
cmd/main.go	Ponto de entrada: leitura de flags, sinal de shutdown e inicialização
internal/crawler	Orquestração geral: cria channels, pool, scheduler e exibe estatísticas
internal/models	Tipos de dados compartilhados: Job, PageResult e Stats
internal/parser	Extração de links de documentos HTML
internal/storage	Conjunto de URLs visitadas, thread-safe
internal/worker	Worker pool com HTTP client e rate limiter
internal/scheduler	Fila de jobs, deduplicação e controle de profundidade

7. Arquitetura e Fluxo de Dados

O sistema segue o padrão **Pipeline com Worker Pool** — um dos padrões clássicos de concorrência em Go. A comunicação entre componentes é feita exclusivamente via **channels**, sem memória compartilhada direta (exceto o **VisitedSet** e o **Stats**, ambos protegidos por primitivas atômicas ou mutex).

7.1 Fluxo principal

- **Scheduler** injeta a URL semente no channel **jobs**.
- **Workers** consomem o channel **jobs**, fazem o HTTP GET, extraem links e devolvem **PageResult** pelo channel **results**.
- **Scheduler** consome o channel **results**, descarta URLs já visitadas e enfileira as novas.
- Quando a fila esvazia e não há jobs em voo, o Scheduler **fecha** o channel **jobs**.
- Os workers encerram ao receber o fechamento do channel e o **pool.Wait()** desbloqueia.

7.2 Channels utilizados

Channel	Tipo	Capacidade	Produtor	Consumidor
jobs	models.Job	= workers	Scheduler	Workers
results	models.PageResult	= workers	Workers	Scheduler
done	struct{}	0 (sync)	goroutine do Scheduler	Crawler.Run

8. Detalhamento dos Componentes

8.1 main.go — Ponto de Entrada

Responsável por parsear as flags de CLI (**-url**, **-depth**, **-workers**, **-rate**), configurar o **context com cancelamento por sinal** (SIGINT/SIGTERM) e delegar a execução ao **crawler.New(cfg).Run(ctx)**.

8.2 crawler.go — Orquestrador

Cria todos os channels e componentes, dispara o Scheduler em uma goroutine separada, aguarda o encerramento via channel **done** e exibe as estatísticas finais. É o único ponto que conhece todos os subsistemas.

8.3 models.go — Tipos de Dados

Tipo	Campos	Descrição
Job	URL string, Depth int	Unidade de trabalho enviada aos workers
PageResult	Job, Links []string, WorkerID int, Err error	Resultado retornado pelo worker ao Scheduler

Tipo	Campos	Descrição
Stats	PagesVisited, PagesFetched, LinksFound, Errors int64	Contadores atômicos de métricas

O tipo **Stats** usa **sync/atomic** para garantir leituras e escritas seguras de múltiplos workers sem mutex.

8.4 parser.go — Extração de Links

Usa o tokenizador de golang.org/x/net/html para percorrer o HTML token a token e extrair atributos **href** de tags **<a>**. A função **resolve()** converte URLs relativas em absolutas e descarta esquemas não-HTTP/HTTPS e fragmentos (**#anchor**).

8.5 visited.go — Conjunto de URLs Visitadas

Implementa um **map[string]struct{}** protegido por **sync.Mutex**. O método **Visit(url)** é atômico: verifica e insere em uma única operação, retornando **true** apenas na primeira vez que a URL é vista.

8.6 worker.go — Pool de Workers e Rate Limiter

Rate Limiter: usa **time.Ticker** com intervalo calculado como **1s / rate**. Workers chamam **Wait(ctx)**, que bloqueia até o próximo tick ou até o contexto ser cancelado.

Worker Pool: cada worker roda em uma goroutine independente, consumindo o channel **jobs**. O cliente HTTP tem timeout de 10 segundos. O User-Agent é identificado como **GoConcurrentCrawler/1.0 (projeto academico)**.

8.7 scheduler.go — Fila e Controle de Profundidade

Mantém uma **fila em memória** (slice de **Job**) e um contador **inFlight** de jobs em processamento. Usa o padrão de **select com canal nulo**: quando não há pending, **sendCh** é **nil** e o case de envio é ignorado automaticamente, evitando busy-wait.

Ao receber um resultado, verifica se **nextDepth <= maxDepth** e só enfileira links novos (não visitados). Quando **len(pending) == 0 && inFlight == 0**, o loop termina e o channel **jobs** é fechado via **defer**.

9. Concorrência e Segurança

Componente	Mecanismo de Proteção	Justificativa
VisitedSet	sync.Mutex	Operação check-and-set deve ser atômica
Stats	sync/atomic	Contadores independentes; atomic é mais eficiente que mutex
jobs channel	Fechamento pelo Scheduler	Sinaliza fim de trabalho aos workers (Go idiomático)
Cancelamento	context.Context	Propagação de shutdown por todos os subsistemas

O projeto **não usa `sync.WaitGroup` para workers diretamente no `main`** — o pool encapsula internamente o `wg.Wait()`. O cancelamento via `context` garante que todos os workers parem de forma ordenada ao receber Ctrl+C.

10. Saída do Programa

10.1 Durante a execução

```
[Worker 3] Crawling https://www.unb.br/ (depth 0)
[Scheduler] https://www.unb.br/ -> 42 links (38 novos enfileirados)
```

10.2 Estatísticas finais

```
===== ESTATÍSTICAS FINAIS =====
URLs únicas descobertas : 128
Páginas baixadas (200)  : 115
Links encontrados       : 3204
Erros                   : 13
Tempo total             : 4.321s
=====
```

11. Dependências

Pacote	Uso	Como obter
golang.org/x/net/html	Tokenização e parsing de HTML	go mod tidy
sync, sync/atomic	Mutex e operações atômicas (stdlib)	Nativo do Go
net/http	Cliente HTTP para buscar páginas (stdlib)	Nativo do Go
context	Propagação de cancelamento (stdlib)	Nativo do Go
time	Rate limiter via Ticker (stdlib)	Nativo do Go

12. Referências

- **Go Tour — Concorrência:** <https://go.dev/tour/concurrency/1>
- **Go Blog — Pipelines e cancelamento:** <https://go.dev/blog/pipelines>
- **CSP — Hoare, C.A.R. (1978):** Communicating Sequential Processes, CACM
- **golang.org/x/net/html:** <https://pkg.go.dev/golang.org/x/net/html>
- **Go Race Detector:** https://go.dev/doc/articles/race_detector